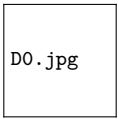
Domain Relational Calculus & QBE

Z2004: Database Management Systems — L14

Dr. Innocent Nyalala



IIT Madras Zanzibar

31 March 2026

Lecture Outline

Domain Relational Calculus — Core Idea

General Form

$$\{\langle x_1, x_2, \ldots, x_k \rangle \mid P(x_1, x_2, \ldots, x_k)\}$$

- $x_1, \ldots, x_k$ are **domain variables** — each ranges over the *values* of one attribute (a column's domain)
- $P(\cdot)$ is a predicate built from atoms, $\wedge$, $\vee$, $\neg$, $\exists$, $\forall$
- Angle brackets $\langle \cdots \rangle$ denote a **list of values** (not a whole tuple)

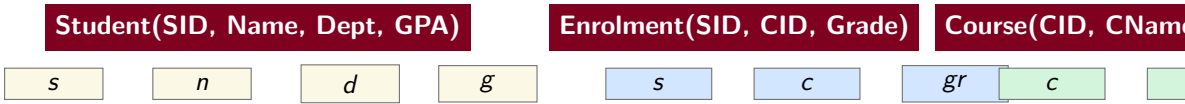
Atoms in DRC:

- $R(x_1, x_2, \ldots)$ — tuple $\langle x_1, x_2, \ldots \rangle$ is in R
- $x\ \theta\ y$ — compare two domain vars
- $x\ \theta\ c$ — compare var with constant

Key difference from TRC:

- TRC: t is an entire *tuple*
- DRC: each x_i is a single *column value*
- DRC is closer to SQL `SELECT` columns directly

Schema Reminder



Domain variables used in examples:

- s = SID, n = Name, d = Dept, g = GPA *(from Student)*
- s = SID, c = CID, gr = Grade *(from Enrolment — same SID domain)*
- c = CID, cn = CName, cr = Credits *(from Course)*

DRC Example 1 — Students with GPA > 3.5

TRC (L13)

$\{t \mid t \in \text{Student} \wedge t.\text{GPA} > 3.5\}$

DRC Equivalent

$$\{\langle s, n, d, g\rangle \mid \text{Student}(s, n, d, g) \wedge g > 3.5\}$$

SQL:

```
SELECT SID, Name, Dept, GPA
FROM   Student
WHERE  GPA > 3.5;
```

Observation

In DRC, each attribute gets its own variable. The atom $\text{Student}(s, n, d, g)$ asserts that the tuple $\langle s, n, d, g\rangle$ exists in the Student relation.

DRC Example 2 — Names of CS Students

TRC (L13)

$\{t.\text{Name} \mid t \in \text{Student} \wedge t.\text{Dept} = \text{`CS'}\}$

DRC Equivalent

$$\{\langle n \rangle \mid \exists s \exists d \exists g\, (\text{Student}(s, n, d, g) \wedge d = \text{`CS'})\}$$

SQL:

```
SELECT Name
FROM   Student
WHERE  Dept = 'CS';
```

Existential Quantification of Unused Variables

Variables s, d, g are needed to "fill in" the Student atom but are not projected. They **must** be bound with $\exists$.

DRC Example 3 — Students Enrolled in C101

TRC (L13)

$$\{t.\text{Name} \mid t \in \text{Student} \wedge \exists e \in \text{Enrolment}(e.\text{SID} = t.\text{SID} \wedge e.\text{CID} = \text{`C101'})\}$$

DRC Equivalent

$$\{\langle n \rangle \mid \exists s \exists d \exists g \exists gr\ (\text{Student}(s, n, d, g) \wedge \text{Enrolment}(s, \text{`C101'}, gr))\}$$

SQL:

```
SELECT DISTINCT s.Name
FROM   Student s  JOIN  Enrolment e  ON s.SID = e.SID
WHERE  e.CID = 'C101';
```

Join via Shared Variable

Notice that the *same* variable s appears in both Student$(s, \ldots)$ and Enrolment$(s, \ldots)$ — this implicitly enforces the join condition Student.SID = Enrolment.SID.

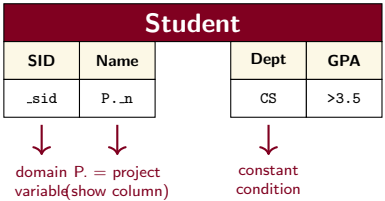
DRC vs. TRC — Head-to-Head Comparison

Dimension	**TRC**	**DRC**
Variable type	Whole tuple t	Individual column values x_i
Membership atom	$t \in R$	$R(x_1, x_2, \ldots, x_n)$
Attribute access	t.GPA	Direct variable g
Projection	Prefix of result: t.Name	Angle-bracket list: $\langle n \rangle$
Unused attrs	Implicit (part of tuple)	Must be $\exists$-quantified
Verbosity	Less verbose	More verbose for wide relations
SQL closeness	Moderate	High (mirrors column-by-column)
Expressive power	Identical (Codd's Theorem)	Identical
Safety concept	Domain of formula	Same concept

QBE — Query By Example

What is QBE?

- Invented at IBM Research by Moshé Zloof, 1977
- **Visual / form-based** query interface built on DRC
- User fills in a grid (skeleton table); the system evaluates the query
- Predecessor of modern GUI query builders (MS Access, etc.)
- No need to learn formal query syntax



QBE Notation Rules

Key QBE Notation

- **P.** prefix — "Print" (project) this column
- **_var** — domain variable (underscore prefix)
- Plain constant (e.g. `CS`) — equality condition
- Operator prefix (e.g. `>3.5`) — range condition
- Same **_var** in multiple tables — join condition
- **NOT** keyword — negation

QBE for: "Names of CS students with GPA ¿ 3.5"

SID	Name	Dept	GPA
	`P._n`	`CS`	`>3.5`

QBE join: "Names enrolled in C101"

Enrolment		
SID	**CID**	**Grade**
`_s`	`C101`	

Student	
SID	**Name**
`_s`	`P._n`

QBE — Strengths, Limitations, Legacy

Strengths of QBE

- Very intuitive for non-technical users
- Visual: no need to remember keywords
- Direct mapping to DRC semantics
- Supports selection, projection, join, negation
- Influenced MS Access query designer

Limitations of QBE

- Difficult to express complex nested queries
- Aggregation is cumbersome
- Not standardised across vendors
- Screen real estate grows with table width
- Superseded by SQL in most products

QBE Legacy in Modern Tools

MS Access "Design View", Tableau's drag-and-drop, Power Query graphical steps — all descend from QBE's idea of *showing the user a skeleton and letting them fill in conditions*.

Summary

Key Takeaways — L14 Domain Relational Calculus & QBE

1. DRC uses *domain variables* (one per attribute) instead of tuple variables
2. Atom form: $R(x_1, \ldots, x_n)$; join = sharing a variable across relations
3. Unused variables must be bound with $\exists$
4. DRC $\equiv$ TRC $\equiv$ RA (Codd's Theorem — same expressive power)
5. **QBE**: IBM's grid-based visual query language grounded in DRC
6. QBE key notation: `P.` (project), `_var` (domain variable), constants and operators for conditions

Next Lecture — L15: SQL Introduction: DDL

L15 (1 Apr): We move from theory to practice — SQL history, T-SQL data types, `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`, constraints, and the difference between `TRUNCATE` and `DROP`.